Master Big-O Notation

Complete Student Handout: Time & Space Complexity in Python

🚀 The Big Idea: What is Big-O?

When software engineers write code, they don't just care if it works—they care about how it **scales**. **Big-O Notation** is a mathematical framework that describes how an algorithm behaves as the input size (n) grows towards infinity.

> **THE TWO DIMENSIONS OF BIG-O**
>
> - ⏱️ **Time Complexity:** Measures how the **number of computational steps** increases as input size n grows.
> - 💾 **Space Complexity:** Measures how much **extra RAM** / **memory** the algorithm allocates as input size n grows.

*Note: Big-O does not measure actual seconds or megabytes, because faster hardware runs code quicker. Instead, Big-O measures the **rate of growth** relative to input size n (e.g., list length, string length, or dictionary keys).*

EFFICIENCY SPECTRUM (FASTEST → SLOWEST)

O(1)	O(n)	$O(n^2)$	$O(2^n)$
Constant	Linear	Quadratic	Exponential

🔍 The Big-O Lineup: Step-by-Step Breakdown

1️⃣ $O(1)$ — Constant Complexity GOLD STANDARD

Time Complexity $O(1)$: The code takes the exact same number of steps regardless of whether n is 10 or 10,000,000.
Space Complexity $O(1)$: Memory usage stays constant. No new large structures are created in RAM.

```python
# 1. Direct index access — Time: O(1) | Space: O(1)
first_item = nums[0]

# 2. Basic math formula — Time: O(1) | Space: O(1)
result = 100 * 5

# 3. Hash lookup (Sets and Dicts are instant!) — Time: O(1) | Space: O(1)
if "bob" in users_set:
    print("Found!")
```

2️⃣ $O(n)$ — Linear Complexity FAIR & PREDICTABLE

Time Complexity $O(n)$: Work grows in direct proportion to n. Double the input size, double the steps.
Space Complexity: $O(1)$ if iterating without creating a new copy; $O(n)$ if constructing a new collection of size n.

```python
# Example A: O(n) Time | O(1) Space (No extra RAM created)
for x in nums:
    print(x)

# Searching a LIST (must inspect items sequentially) — Time: O(n) | Space: O(1)
if "bob" in users_list:
    print("Found!")

# Example B: O(n) Time | O(n) Space (Creates brand-new list holding 'n' items)
doubled_nums = [x * 2 for x in nums]
```

3️⃣ $O(n^2)$ — Quadratic Complexity DANGEROUS AT SCALE

Time Complexity $O(n^2)$: Work grows like a square. If input size doubles, steps **quadruple** ($2^2 = 4$). If n grows by $10 \times$, steps explode by $100 \times$.
Space Complexity: $O(1)$ if comparing in-place; $O(n^2)$ if creating an $n \times n$ 2D grid/matrix.

```
# Example A: O(n²) Time | O(1) Space (Nested loops over same data)
for item1 in tickets:
    for item2 in tickets:
        if item1 == item2:
            print("Duplicate found!")

# Example B: O(n²) Space (Creates n x n matrix in RAM)
grid = [[i * j for j in range(n)] for i in range(n)]
```

4️⃣ $O(2^n)$ — Exponential Complexity THE ULTIMATE NIGHTMARE

Time Complexity $O(2^n)$: Every single extra item added to input **doubles** total work.
Space Complexity $O(n)$: Memory is consumed by the recursive call stack reaching depth n.

```
# Classic Naive Recursive Fibonacci — Time: O(2ⁿ) | Space: O(n)
def fibonacci(num):
    if num <= 1:
        return num
    return fibonacci(num - 1) + fibonacci(num - 2)
```

🎴 Cheat Sheet: Code Spotting Matrix

Big-O	Growth Rate	Code Pattern to Look For	Real-World Analogy
$O(1)$	Constant	Direct lookup, math operation, checking `set` or `dict`	Peeking at the top ticket in a stack
$O(n)$	Linear	Single loop, or list scans (`in list`, `min()`, `max()`)	Reading every single ticket in a stack once
$O(n^2)$	Quadratic	Nested loops (loop running inside another loop)	Comparing every ticket against every other ticket
$O(2^n)$	Exponential	Double recursion (function calling itself twice per step)	Finding every possible combination of tickets

🧠 The Cheat Code: How to Determine Big-O

1. **Count Loops & Data Structures:**
   - *Time:* No loops = $O(1)$ | 1 loop = $O(n)$ | 2 nested loops = $O(n^2)$
   - *Space:* If creating a new structure that grows with n, space is $O(n)$.
2. **Drop Constants:** Big-O measures growth shape, not exact step totals.
   $O(n) + O(n) = O(2n) \rightarrow$ **Drop the 2** $\rightarrow O(n)$ Time.
3. **Keep Only the Worst-Case Term:** If code has an $O(1)$ step, an $O(n)$ loop, and an $O(n^2)$ loop, drop the smaller parts. Total complexity is $O(n^2)$.

💡 MENTAL STRESS-TEST CHECKLIST

- **Time Check:** *"If input size jumps from 10 to 1,000 (100 times larger), does execution time stay the same ($O(1)$), increase 100 times ($O(n)$), or explode $10{,}000 \text{ times}$ ($O(n^2)$)?"*
- **Space Check:** *"Does my code allocate new lists, sets, or dicts whose memory footprint grows as n grows?"*